

Hospitals and medical facilities

```
#include <iostream>

#include <vector>

#include <map>

#include <set>

#include <algorithm>

#include <thread>

#include <chrono>

using namespace std;

// ANSI Colors

class Style {

public:

    static constexpr const char* HEADER = "\033[95m";

    static constexpr const char* BLUE  = "\033[94m";

    static constexpr const char* CYAN  = "\033[96m";

    static constexpr const char* GREEN = "\033[92m";

    static constexpr const char* YELLOW = "\033[93m";

    static constexpr const char* RED   = "\033[91m";

    static constexpr const char* BOLD  = "\033[1m";

    static constexpr const char* RESET = "\033[0m";

};

class MedicalExpertSystem {

private:

    map<string, vector<string>> knowledge_base;

    vector<string> patient_symptoms;

public:

    MedicalExpertSystem() {

        // Knowledge Base

        knowledge_base = {

            {"Common Cold", {"cough", "runny_nose", "sneezing", "sore_throat", "mild_fever"}},

            {"Influenza (Flu)", {"high_fever", "chills", "body_ache", "headache", "fatigue"}},
```

```

        {"COVID-19", {"fever", "dry_cough", "loss_of_taste_or_smell", "difficulty_breathing",
"tiredness"}},

        {"Food Poisoning", {"nausea", "vomiting", "diarrhea", "abdominal_pain", "mild_fever"}},

        {"Dengue", {"high_fever", "severe_headache", "pain_behind_eyes", "joint_pain", "skin_rash"}}

    };
}

void print_header() {
    cout << Style::CYAN << Style::BOLD;

    cout << " _____\n";
    cout << "|         MEDICAL DIAGNOSIS EXPERT SYSTEM         |\n";
    cout << "| _____|\n";

    cout << Style::RESET;

    cout << Style::YELLOW << "Instructions:" << Style::RESET
        << " Please answer the following symptom checks.\n";

    cout << string(58, '-') << "\n\n";
}

vector<string> get_all_unique_symptoms() {
    set<string> all;

    for (auto &entry : knowledge_base) {
        for (auto &sym : entry.second) {
            all.insert(sym);
        }
    }

    return vector<string>(all.begin(), all.end());
}

void ask_questions() {
    print_header();

    vector<string> symptoms = get_all_unique_symptoms();

    for (auto &symptom : symptoms) {
        string formatted = symptom;

        replace(formatted.begin(), formatted.end(), '_', ' ');
    }
}

```

```

while (true) {

    cout << Style::BLUE << "> " << Style::RESET

        << "Do you have " << Style::BOLD << formatted

        << Style::RESET << "? (y/n): ";

    string response;

    cin >> response;

    transform(response.begin(), response.end(), response.begin(), ::tolower);

    if (response == "y" || response == "yes") {

        patient_symptoms.push_back(symptom);

        break;

    }

    else if (response == "n" || response == "no") {

        break;

    }

    else {

        cout << Style::RED

            << "[!] Invalid input. Please type 'y' or 'n'.\\n"

            << Style::RESET;

        }

    }

}

void simulate_processing() {

    cout << "\\n" << Style::YELLOW << "Processing Data" << Style::RESET;

    for (int i = 0; i < 3; i++) {

        this_thread::sleep_for(chrono::milliseconds(400));

        cout << ".";

    }

    cout << "\\n\\n";

}

void diagnose() {

```

```

simulate_processing();

map<string, double> diagnosis_scores;

for (auto &entry : knowledge_base) {
    string disease = entry.first;
    vector<string> symptoms = entry.second;
    int matches = 0;
    for (auto &ps : patient_symptoms) {
        if (find(symptoms.begin(), symptoms.end(), ps) != symptoms.end()) {
            matches++;
        }
    }

    double score = ((double)matches / symptoms.size()) * 100;
    diagnosis_scores[disease] = score;
}

// Find max score
string likely_disease;
double max_score = -1;
for (auto &entry : diagnosis_scores) {
    if (entry.second > max_score) {
        max_score = entry.second;
        likely_disease = entry.first;
    }
}

display_results(likely_disease, max_score);
}

void display_results(string disease, double score) {
    cout << Style::GREEN << Style::BOLD;
    cout << "_____\\n";
    cout << "|          DIAGNOSIS REPORT          |\\n";
    cout << "_____\\n";
    cout << Style::RESET;
}

```

```

if (score == 0) {
    cout << "\nCondition: No matching disease found.\n";
    cout << "Match: 0.0%\n";
    cout << "Advice: Consult a doctor.\n";
}
else if (score >= 60) {
    cout << "\nCondition: " << Style::RED << disease << Style::RESET << "\n";
    cout << "Match: " << Style::RED << score << "%" << Style::RESET << "\n";
    cout << "Advice: Seek medical attention.\n";
}
else {
    cout << "\nCondition: " << Style::CYAN << disease << Style::RESET << "\n";
    cout << "Match: " << Style::CYAN << score << "%" << Style::RESET << "\n";
    cout << "Advice: Rest and monitor symptoms.\n";
}
cout << "\n" << string(58, '-') << "\n";
}
};

int main() {
    try {
        MedicalExpertSystem system;
        system.ask_questions();
        system.diagnose();
    }
    catch (...) {
        cout << Style::RED << "[!] System terminated by user." << Style::RESET << endl;
    }
    return 0;
}

```

Airline scheduling and cargo schedules

```
#include <iostream>

#include <map>

#include <vector>

#include <string>

using namespace std;

// Function to get flight schedule
void get_flight_schedule(string flight_type, string destination,
                        float cargo_weight, string preferred_time) {

    // Flight schedules
    map<string, vector<string>> schedules = {
        {"Morning", {"06:00", "07:30", "09:00"}},
        {"Afternoon", {"12:00", "14:00", "15:30"}},
        {"Evening", {"18:00", "19:30", "21:00"}}
    };

    // Cargo limits
    map<string, int> cargo_limits = {
        {"Passenger", 200},
        {"Cargo", 1000}
    };

    // Rule 1: Validate Cargo Weight
    if (cargo_weight > cargo_limits[flight_type]) {
        cout << "\n Scheduling Result" << endl;
        cout << "-----" << endl;
        cout << " Status      : Rejected" << endl;
        cout << " Reason      : Cargo exceeds limit for "
```

```

        << flight_type << " flight ("
        << cargo_limits[flight_type] << " kg)" << endl;
    cout << " Suggestion      : Switch to Cargo flight or reduce cargo weight." << endl;
    return;
}

```

```

// Rule 2: Match Preferred Time

```

```

if (schedules.find(preferred_time) == schedules.end()) {
    cout << "\n Scheduling Result" << endl;
    cout << "-----" << endl;
    cout << " Status      : Failed" << endl;
    cout << " Reason      : Invalid preferred time slot." << endl;
    cout << " Suggestion   : Choose Morning, Afternoon, or Evening." << endl;
    return;
}

```

```

// Rule 3: Assign First Available Slot

```

```

string assigned_slot = schedules[preferred_time][0];

```

```

// Success Output

```

```

cout << "\n Scheduling Result" << endl;
cout << "-----" << endl;
cout << " Status      : Success" << endl;
cout << " Flight Type   : " << flight_type << endl;
cout << " Destination   : " << destination << endl;
cout << " Assigned Time  : " << assigned_slot << endl;
cout << " Cargo Weight   : " << cargo_weight << " kg" << endl;
cout << " Note          : Flight scheduled successfully!" << endl;
}

```

```

// ---- MAIN PROGRAM ----

```

```
int main() {

    string flight_type, destination, preferred_time;
    float cargo_weight;

    cout << "\n Airline Scheduling and Cargo Expert System" << endl;
    cout << "-----" << endl;

    cout << "Enter flight type (Passenger/Cargo): ";
    cin >> flight_type;

    cout << "Enter destination: ";
    cin >> destination;

    cout << "Enter cargo weight (in kg): ";
    cin >> cargo_weight;

    cout << "Preferred time (Morning/Afternoon/Evening): ";
    cin >> preferred_time;

    // Function Call
    get_flight_schedule(flight_type, destination,
                        cargo_weight, preferred_time);

    return 0;
}
```